

Khi AI có thể viết code chỉ từ một vài dòng mô tả, môn **Kỹ thuật lập trình** không còn là việc dạy sinh viên cách "gõ cho đúng cú pháp" (Syntax-heavy) mà chuyển sang dạy cách **"thiết kế giải pháp và kiểm soát logic"** (Logic & Architecture-heavy).

Dưới đây là phương pháp giảng dạy giúp sinh viên không bị "lười hóa" bởi AI mà trở thành những kỹ sư làm chủ được AI:

1. Thay đổi tư duy: Từ "Coder" sang "Architect" (Kiến trúc sư)

Giảng viên nên dạy sinh viên rằng việc AI viết được code giống như việc ta có một thợ xây rất nhanh, nhưng người thợ đó không biết ngôi nhà nên xây thế nào để không đổ.

- **Phân rã bài toán (Problem Decomposition):** Tập trung dạy cách chia nhỏ một bài toán lớn thành các module, hàm và class. AI có thể viết code cho một hàm, nhưng thiết kế mối quan hệ giữa các hàm là việc của con người.
- **Kỹ năng Prompt Engineering trong lập trình:** Dạy sinh viên cách mô tả logic thuật toán bằng ngôn ngữ tự nhiên một cách chặt chẽ để AI có thể thực thi chính xác.

2. Ưu tiên "Đọc và Phê bình code" thay vì "Viết code từ đầu"

Trong thực tế, lập trình viên hiện nay dành nhiều thời gian để sửa code AI hơn là tự viết.

- **Code Review:** Đưa cho sinh viên một đoạn code do AI tạo ra (có thể chứa lỗi bảo mật, lỗi logic hoặc hiệu suất kém) và yêu cầu họ tìm lỗi, tối ưu hóa hoặc giải thích cách hoạt động.
- **Unit Testing:** Dạy sinh viên viết bài kiểm thử (Test cases) trước khi yêu cầu AI viết code. Nếu code AI chạy sai Test case, sinh viên phải biết tại sao.

3. Tập trung vào "Vibe Coding" và Trực quan hóa

Tận dụng khả năng tạo code nhanh của AI để sinh viên thấy được kết quả ngay lập tức, từ đó tạo hứng thú:

- **Học thông qua sản phẩm:** Thay vì giải các bài toán "tính tổng mảng" khô khan, hãy để sinh viên dùng AI hỗ trợ viết các ứng dụng nhỏ (Mini-game, quản lý sinh viên có giao diện, công cụ vẽ đồ thị) ngay từ những tuần đầu.
- **Sử dụng sơ đồ khối (Flowchart):** Yêu cầu sinh viên vẽ sơ đồ tư duy hoặc sơ đồ luồng dữ liệu trước khi chạm vào bàn phím. Điều này đảm bảo sinh viên hiểu logic lỗi trước khi nhờ AI hiện thực hóa.

4. Đánh giá năng lực: Chống "copy-paste" thụ động

Để đảm bảo sinh viên thực sự hiểu bài, phương pháp kiểm tra cần thay đổi:

- **Vấn đáp trực tiếp:** Sau khi sinh viên nộp bài (có thể dùng AI hỗ trợ), giảng viên hỏi ngẫu nhiên một dòng code: "Tại sao chỗ này lại dùng vòng lặp này?", "Nếu thay đổi tham số này thì điều gì xảy ra?".
- **Kiểm tra trên giấy/Offline:** Duy trì một tỷ lệ nhất định các bài kiểm tra thực hiện tại lớp không có internet để củng cố kiến thức nền tảng (cấu trúc dữ liệu, giải thuật cơ bản).
- **Đề bài mở và thực tế:** Đưa ra những yêu cầu mà AI hiện tại khó có thể giải quyết trọn vẹn nếu không có sự can thiệp chi tiết về nghiệp vụ cụ thể.

5. Giảng dạy về Đạo đức và Hiệu suất

- **Bản quyền và Bảo mật:** Nhắc nhở sinh viên về việc đưa dữ liệu nhạy cảm lên AI và nguồn gốc của mã nguồn mà AI gợi ý.
- **Độ phức tạp thuật toán (Big O):** AI có thể viết code chạy được, nhưng thường không tối ưu. Giảng viên cần nhấn mạnh việc phân tích hiệu năng để sinh viên không chấp nhận những đoạn code "rác" hay chạy chậm từ AI.

Cốt lõi là: Giảng viên phải biến AI thành một "trợ giảng 24/7" cho sinh viên, đồng thời đóng vai trò là người thẩm định chất lượng cuối cùng.